

A Layered, Dependency-Light Architecture for Streaming LLM Chat Clients on Android: Design and Engineering Trade-offs in AI Chat Assistant

Kaustubha Venkata Eluri
Northeastern University

Abstract—This paper presents a technical case study of AI Chat Assistant, a native Android chat client for OpenAI-compatible large language model (LLM) endpoints, built as a single-Activity, clean-architecture Java application. The system supports server-sent-event (SSE) streaming responses, Room-backed local conversation history, runtime model switching, and a built-in offline mock provider that replays the same streaming state machine as the live network path. The core technical contribution of this work is not a novel algorithm but a set of deliberate, documented architectural decisions made to keep a small Android codebase maintainable without adopting heavyweight frameworks: a manual service-locator dependency injection scheme instead of Hilt/Dagger, a dual HTTP strategy that uses raw OkHttp for token-level SSE streaming while reserving Retrofit for non-streaming requests, and a sealed `Resource<T>` type that replaces ad hoc boolean-based loading/error/success state with an exhaustive state machine. We describe the system’s layered architecture, walk through nine architecture decision records (ADRs) that document why each trade-off was made and under what conditions it should be revisited, and honestly report the project’s limitations (no encryption at rest, no request cancellation, sparse automated test coverage) rather than claiming a level of rigor the project does not have. No formal quantitative evaluation (e.g., latency benchmarks, user studies) was conducted; we frame this paper as an engineering design and decision-rationale case study rather than an empirical systems-performance paper, and we explicitly describe what a rigorous evaluation of this system would require.

Index Terms—Android architecture, large language models, streaming APIs, mobile software engineering, dependency injection, clean architecture, architecture decision records

I. INTRODUCTION

Consumer and developer-facing chat interfaces to large language models (LLMs) have become a common integration surface for mobile applications, typically built atop OpenAI-compatible `/chat/completions` HTTP APIs. Building a mobile client for such an API surfaces a set of recurring engineering problems that are somewhat distinct from generic mobile app development: (1) the API’s most useful mode, server-sent-event streaming, is poorly served by request/response abstractions that buffer entire HTTP bodies before returning; (2) UI states must track an asynchronous, multi-stage lifecycle (idle, sending, receiving partial tokens, complete, or failed) that is easy to represent incorrectly with independent boolean flags; (3) development and demoing of the UI should not require a live, metered API key; and (4) at small-to-medium codebase scale, adopting industrial dependency-injection frameworks such as Hilt or Dagger

carries real overhead (build time, generated code, annotation ceremony) whose benefit is not obviously worth the cost.

This paper documents AI Chat Assistant, an Android chat client built to explore and resolve these problems concretely, and treats the resulting codebase as a case study in software architecture decision-making at a deliberately small scale. Rather than presenting benchmark results, we present the problem context, the chosen architecture, the specific decisions made (with their justifications and their explicitly acknowledged costs, as recorded contemporaneously in the project’s architecture decision records), and the limitations of the resulting system. The motivation for this framing is intellectual honesty: the project has no user base, no load testing, and no controlled evaluation, and a paper that manufactured such results would misrepresent the work. Instead, we argue that documenting engineering trade-offs rigorously and honestly is itself a useful contribution, particularly for small-team or solo-developer contexts where the “correct” architecture is not the one used at FAANG scale, but the one that matches the actual size of the problem.

II. RELATED WORK AND BACKGROUND

A. Streaming LLM APIs

Most hosted LLM providers expose an OpenAI-compatible `/chat/completions` endpoint that, when invoked with a `stream: true` parameter, returns a response as a sequence of server-sent events (SSE), each carrying an incremental token or set of tokens, terminated by a sentinel `data: [DONE]` event. Because token-by-token delivery substantially improves perceived latency in interactive chat interfaces, correctly plumbing SSE end-to-end through a mobile client’s networking and UI layers is a standard, if under-documented, engineering requirement for any LLM-backed chat app. This differs materially from typical mobile REST integration, where a client library that returns a fully materialized response body is sufficient.

B. Mobile Architecture Patterns

Android application architecture has converged, over roughly a decade, on a small number of recurring patterns: separation into UI, domain, and data layers (often called “clean architecture” in the Android community, following the general layering principle popularized by Robert C. Martin); a `ViewModel` holding UI-observable state, historically via

LiveData and more recently via Kotlin Flow; and dependency injection, most commonly via Dagger/Hilt on Android, to manage the construction and lifetimes of layer boundaries. A recurring, less-discussed question is when the overhead of a compile-time DI framework such as Hilt is justified relative to a hand-written composition root (often termed a "service locator" or "poor man's DI container"). This project takes an explicit position on that question, discussed in Section V.

C. Asynchronous State Modeling

A common anti-pattern in UI state management is representing an asynchronous operation's status via multiple independent boolean or nullable fields (e.g., `isLoading`, `errorMessage`, `data`), which admits states that should be unreachable (e.g., loading and error simultaneously true). Sealed/algebraic state types — a `Resource<T>` or `Result<T>` wrapper with exhaustive `Loading`, `Success`, and `Error` variants — are a widely used alternative in both Android and iOS communities to make invalid states unrepresentable at the type level. This project adopts that pattern deliberately, as detailed in Section V.

We do not cite specific academic papers for these background claims because they reflect widely known, general software-engineering practice rather than results attributable to a specific publication; fabricating citations to non-existent papers would misrepresent the state of prior work.

III. SYSTEM DESIGN AND ARCHITECTURE

AI Chat Assistant is a native Android application written in Java 17, targeting a minimum SDK of 24 (Android 7.0) and a compile SDK of 34 (Android 14). The application consists of 31 Java source files organized into three architectural layers plus a cross-cutting composition layer.

A. Layering

The system is organized top-to-bottom as:

- **UI layer:** `MainActivity` (the single host activity), `ChatFragment` (the sole primary screen), `ChatAdapter` (a `RecyclerView` adapter with three view types: user bubble, AI bubble, typing indicator), `ChatViewModel` and `ChatViewModelFactory`, and `SettingsBottomSheet` (a Material 3 bottom-sheet dialog for runtime configuration).
- **Domain layer:** a `ChatRepository` interface, use cases (`SendMessageUseCase`, `ObserveMessagesUseCase`, `ClearChatUseCase`), and a plain `Message` domain model.
- **Data layer:** `ChatRepositoryImpl`, which composes `RemoteChatDataSource` (Retrofit/OkHttp-backed real API client), `MockStreamingDataSource` (an offline streaming simulator), and `ChatDatabase/MessageDao/MessageEntity` (Room persistence), plus request/response DTOs (`ChatRequest`, `ChatResponse`, `ChatChoice`).

- **Cross-cutting:** `AIChatApp` (the Application subclass that bootstraps the composition root), `ServiceLocator` (manual dependency injection), `AppConfig` (build-time configuration surface), `Resource<T>` (the async state wrapper), `NetworkUtils` (connectivity checks), `PromptBuilder` (system-prompt and history composition), and `DateUtils`.

The UI layer communicates with the domain layer exclusively via `LiveData<Resource<T>>` streams exposed by `ChatViewModel`; the domain layer's `ChatRepository` interface is the sole abstraction boundary the UI depends on, and it is implemented once, by `ChatRepositoryImpl`, which internally selects between the mock and real data sources based on a runtime "use mock provider" flag and the presence of a configured API key. The UI is never aware of which data source served a given response.

Three structural invariants are enforced by convention across the codebase: (1) fragments and activities contain presentation logic only — no direct Retrofit, Room, or JSON-parsing calls; (2) `ChatRepository` is the single seam between UI and data, so swapping the real network client for the mock provider requires no UI-layer change; and (3) all color and theme decisions live in `res/values/colors.xml` and `themes.xml`, with no `Color.parseColor(...)` literals scattered through Java code.

Figure 1 shows the UI layer's primary screen, `ChatFragment`, rendering a streamed assistant response.

B. Streaming Pipeline

When a user sends a message, `ChatViewModel` invokes `SendMessageUseCase`, which calls into `ChatRepository`. The repository opens either a real SSE connection (via `RemoteChatDataSource`) or a simulated one (via `MockStreamingDataSource`), and emits a sequence of `Resource` states: an initial `Resource.Loading`, followed by partial-text updates as SSE chunks arrive, and finally a `Resource.Success` carrying the completed assistant message, which is persisted via `MessageDao` and re-observed by the UI to refresh the message list. `RemoteChatDataSource` parses the OpenAI-compatible SSE payload line by line, treating each data: {...} chunk as a partial-token emission and the terminal data: [DONE] line as stream completion.

C. Persistence

Room persistence is backed by a single messages table:

```
CREATE TABLE messages (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  role TEXT NOT NULL, -- 'user' | 'assistant' | '  
    system'  
  content TEXT NOT NULL,  
  timestamp INTEGER NOT NULL -- epoch millis  
);
```

`MessageDao` exposes an `observeAll()` query returning `LiveData<List<MessageEntity>>`, so the UI updates reactively on every insert, and a `clear()` method used

by the action-bar “clear chat” affordance. Because the app currently supports only a single ongoing conversation, there is no separate `chats` table; this is a deliberate, documented scope limitation (Section VI) rather than an oversight.

D. Configuration Surface

Runtime-tunable parameters — the system prompt, the API key, the selected model, and the mock-provider toggle — are exposed through `SettingsBottomSheet` and persisted in `SharedPreferences`. Build-time defaults (`OPENAI_API_KEY`, `OPENAI_BASE_URL`, `DEFAULT_MODEL`) are injected via Gradle `buildConfigFields` and surfaced to application code through an `AppConfig` object, keeping `BuildConfig` references out of the rest of the codebase.

IV. KEY ENGINEERING DECISIONS

The project maintains an append-only architecture decision record (ADR) log; this section summarizes the decisions most relevant to the system’s technical identity, including the stated rationale and the cost each decision was explicitly understood to carry at the time it was made.

A. Manual Service Locator Instead of Hilt/Dagger (ADR-001)

A singleton `ServiceLocator` lazily constructs the `OkHttp` client, `Retrofit` instance, `Room` database, and repositories. The stated rationale is that the application’s dependency graph is small (on the order of ten objects), and that Hilt’s annotation-processing build-time cost and generated-code ceremony (`@HiltAndroidApp`, `@AndroidEntryPoint`) are not justified at this scale; a single, top-to-bottom-readable composition file is judged easier to reason about and debug. The acknowledged cost is that `ServiceLocator` is global mutable state, which complicates substitution under test; this is mitigated with a `@VisibleForTesting`-annotated `setForTest(...)` escape hatch rather than solved architecturally.

B. Single-Activity, Single-Fragment Topology (ADR-002)

`MainActivity` hosts exactly one `ChatFragment`, with no Jetpack Navigation graph and no fragment back-stack, on the grounds that a one-screen-plus-bottom-sheet application does not warrant a navigation component. The ADR explicitly names its own obsolescence condition: when the roadmap’s conversation-list feature (Section VII) is implemented, the app becomes multi-screen and Jetpack Navigation will be introduced at that point.

C. Mock Provider Streams the Same Pipeline as the Real One (ADR-003)

`MockStreamingDataSource` does not return a canned response string atomically; it emits synthetic tokens through the identical `Resource.Loading` → `Resource.Success` pipeline used by the real network path. This means the typing-indicator animation, partial-render behavior, and final bubble collapse are all exercised offline, at zero API cost, with no network dependency. The

design rationale explicitly frames this as a UX-correctness measure: a mock that resolves atomically would let bugs in the streaming-specific UI states go undetected until the real API path was exercised.

D. OkHttp Direct for Streaming; Retrofit Only for Non-Streaming (ADR-004)

The streaming request path (`RemoteChatDataSource.streamChatCompletion`) uses raw `OkHttp` calls with manual SSE line-parsing, rather than `Retrofit`’s `Call.execute()`, because `Retrofit`’s default call adapter fully buffers the response body before returning it to the caller — which would defeat the purpose of `stream: true` by delaying the entire response until generation completes. `Retrofit` is retained for the request-building convenience it offers on non-streaming call shapes. The acknowledged cost is that the data-source layer now contains two distinct HTTP invocation styles; this is contained by keeping both within `RemoteChatDataSource` and exposing a single, uniform interface to the rest of the app through `ChatRepository`.

E. Resource<T> Over Independent Boolean Flags (ADR-005)

A hand-written sealed-style wrapper (`Resource.loading()`, `Resource.success(data)`, `Resource.error(message)`) is used for every asynchronous `LiveData` stream in the app, rather than a view-model exposing separate `isLoading`, `error`, and `data` fields. The stated motivation is that independent booleans admit invalid combinations (e.g., loading and error both true simultaneously), whereas the wrapper’s three-variant shape makes such states unrepresentable. The project’s documentation notes this mirrors a `ViewState<T>` pattern used in an iOS sibling project by the same author, suggesting the pattern is treated as a portable design habit rather than an Android-specific idiom.

F. API Key Never Hardcoded (ADR-006)

`app/build.gradle` reads `OPENAI_API_KEY` from `~/.gradle/gradle.properties` or the environment, falling back to an empty string so the project still builds and defaults to the mock provider when no key is configured. Notably, this ADR’s status is recorded as “Accepted (after a key-leak incident)”: an earlier revision of the project had a real API key literally embedded in `build.gradle`, which was caught and rotated before a public push, and the ADR documents the corrective policy going forward, along with `.gitignore` rules excluding `local.properties`, `*.keys`, `secrets.properties`, and related files. We report this candidly as an example of a real security near-miss and its remediation, rather than omitting it to present a cleaner narrative.

G. *ViewBinding Everywhere (ADR-007)*

All view lookups go through generated `*Binding` classes rather than `findViewById`, trading a small amount of generated-code overhead for compile-time null-safety and elimination of runtime `ClassCastException` risk from ID misuse.

H. *Java 17, Not Kotlin (ADR-008)*

The codebase is pure Java, targeting `sourceCompatibility VERSION_17`. The rationale given is historical/contextual — the project began in a Java-heavy course setting — rather than a considered preference for Java over Kotlin on technical grounds; the ADR notes that Java 17’s records, switch expressions, and pattern matching cover much of the ergonomic gap, but also explicitly flags that adopting Jetpack Compose (a roadmap item) would force a Kotlin migration, since Compose is Kotlin-only.

I. *Mock Toggle as a Runtime Setting, Not a Build Variant (ADR-009)*

The mock-provider toggle lives in `SharedPreferences`, controlled from the settings bottom sheet, rather than being implemented as a separate Gradle build variant (e.g., `mockDebug`). This allows a single installed APK to switch between mock and live modes without a reinstall, at the cost of one boolean preference read per outgoing request, which the ADR characterizes as negligible.

J. *Synthesis*

Read together, these decisions share a consistent underlying philosophy: match architectural investment to actual project scale, prefer mechanisms that are easy to delete or replace later over premature adoption of heavyweight frameworks, and treat security and correctness lapses (the API key incident; the potential for invalid UI states) as occasions to record a durable policy rather than a one-off fix.

V. EVALUATION

No formal, controlled evaluation of AI Chat Assistant has been conducted, and we do not report latency numbers, throughput figures, user-study results, or crash-rate statistics, because none were measured. Reporting such numbers without having gathered them would misrepresent the project.

What has been done, according to the project’s own quality gates, is the following manual and automated verification:

- `./gradlew assembleDebug` builds successfully against API level 34 with JDK 17.
- `./gradlew lintDebug` runs clean.
- `./gradlew testDebugUnitTest` executes the existing (currently sparse) unit-test suite.
- A manual constraint check confirms no `findViewById` calls remain in the codebase (`ViewBinding` is used exclusively) and no `Color.parseColor("#...")` literals appear outside the resource XML files.

- The application was built locally with Gradle 8.5, installed on a headless Pixel 8 (API 34) emulator via `adb install`, and screenshotted via `adb exec-out screencap` to confirm the empty state, a streaming mock-provider response, and the settings bottom sheet render correctly end-to-end. These captures constitute the project’s only recorded runtime verification evidence.
- A GitHub Actions CI workflow was added alongside this documentation effort; prior development had been verified locally only, without continuous integration.

A rigorous evaluation of a system of this kind would require, at minimum: instrumented UI tests (e.g., Espresso) exercising the full send → streaming-indicator → final-bubble flow and the settings and clear-chat interactions; unit tests with meaningful coverage of `PromptBuilder`’s prompt-composition logic, the `Resource<T>` state-mapping logic, and `MockStreamingDataSource`’s token-emission timing; measured latency and time-to-first-token statistics against a live OpenAI-compatible endpoint under varying network conditions; and some form of usability evaluation with real users interacting with the chat UI. None of these exist for this project today, and we list them here as the concrete gap between the current state and a genuinely evaluated system, consistent with the project’s own limitations documentation.

VI. LIMITATIONS

The project’s own documentation records the following limitations candidly, and we reproduce them here rather than glossing over them.

A. *Feature Scope*

The application supports exactly one conversation screen with no multi-chat list or navigation drawer; no markdown rendering (assistant code blocks, lists, and links render as plain text); no image or vision-model input; no conversation export; no tablet-optimized layout; and no dark-theme option, using `Theme.Material3.Light.NoActionBar` exclusively.

B. *Streaming*

Only OpenAI-style SSE streaming is supported; endpoints using WebSocket or other custom streaming transports would require a new data source. There is no mechanism to cancel an in-progress stream once started — no “stop generating” affordance exists.

C. *Persistence*

All messages live in a single flat Room table with no grouping by conversation, since only one conversation exists at a time. There is no multi-device sync or cloud backup, and the local SQLite database is unencrypted.

D. *Security*

A user-pasted API key is stored, unencrypted, in `SharedPreferences`, readable by anyone with root access to the device. The build-time key path via `gradle.properties` is suitable for local development

only and is not a production secret-management solution. No certificate pinning is implemented; TLS trust relies solely on the system trust store.

E. Networking

There are no automatic retries on transient failures (time-outs, HTTP 503 surface immediately as errors); no request cancellation when the hosting `Fragment` is destroyed, meaning an in-flight stream continues consuming tokens after the user navigates away; and no handling of HTTP 429 `Retry-After` headers.

F. Testing

Unit-test infrastructure (JUnit 4, Espresso) exists but current coverage is sparse; no instrumented UI tests exist for the streaming-indicator animation, the settings bottom sheet, or the clear-chat flow.

G. Build and Delivery

Dependency injection via a manual `ServiceLocator` requires `@VisibleForTesting` shim methods for test substitution, which is workable at the current scale but would become friction if the dependency graph grows substantially. There was no continuous-integration pipeline prior to the commit that introduced this documentation, and there is no automated release/signing pipeline; APK signing and any store submission would be manual today.

VII. CONCLUSION AND FUTURE WORK

AI Chat Assistant demonstrates that a small, single-screen Android LLM chat client can be built with a conventional three-layer architecture, a hand-rolled dependency-injection composition root, and a small number of carefully justified, explicitly documented architectural decisions, without requiring the heavier machinery (Hilt/Dagger, Jetpack Compose, WebSocket-based custom protocols) that a larger or longer-lived project might eventually need. The project’s most transferable contribution is arguably not the code itself but the discipline of maintaining dated, append-only architecture decision records that state not only what was decided but under what future conditions the decision should be revisited — for example, ADR-002’s explicit trigger to adopt Jetpack Navigation once a multi-conversation UI is built, and ADR-008’s acknowledgment that a future Compose migration would force a Kotlin rewrite.

The project’s own roadmap lays out a phased plan for closing the gaps identified in Section VI: Phase 1 introduces a `chats` table and a navigation-drawer-based conversation list, requiring the migration from single-`Fragment` to Jetpack Navigation anticipated by ADR-002; Phase 2 adds Markdown rendering for assistant messages via the Markwon library; Phase 3 migrates the stored API key from plain `SharedPreferences` to `EncryptedSharedPreferences` backed by the Android Keystore; Phase 4 adds image input for vision-capable models; Phase 5 adds a tablet dual-pane layout; Phase 6 adds

light/dark theme switching; Phase 7 hardens networking with request cancellation on `ViewModel.onCleared()` and exponential-backoff retries, including `Retry-After-aware` handling of HTTP 429; Phase 8 builds out unit and instrumented UI test coverage under the newly added CI workflow; and Phase 9 adds conversation export. Longer-term, non-committed ideas include full-text conversation search via Room’s FTS support, voice input, and OpenAI-style function-calling/tool use. The project’s stated non-goals — building a consumer competitor to existing chat applications, and supporting per-message multi-provider mixing within a single conversation — are recorded explicitly to keep the roadmap bounded.

More broadly, this case study suggests that for small Android codebases integrating streaming LLM APIs, the dominant engineering risks are not algorithmic but architectural and operational: correctly modeling asynchronous UI state, avoiding HTTP abstractions that silently defeat streaming, keeping credentials out of source control, and being honest about what has and has not been tested. Future evaluation work on this specific project should prioritize instrumented UI testing and measured performance characterization over further feature expansion, since feature growth without a test safety net increases the risk that the architectural invariants documented here erode over time.

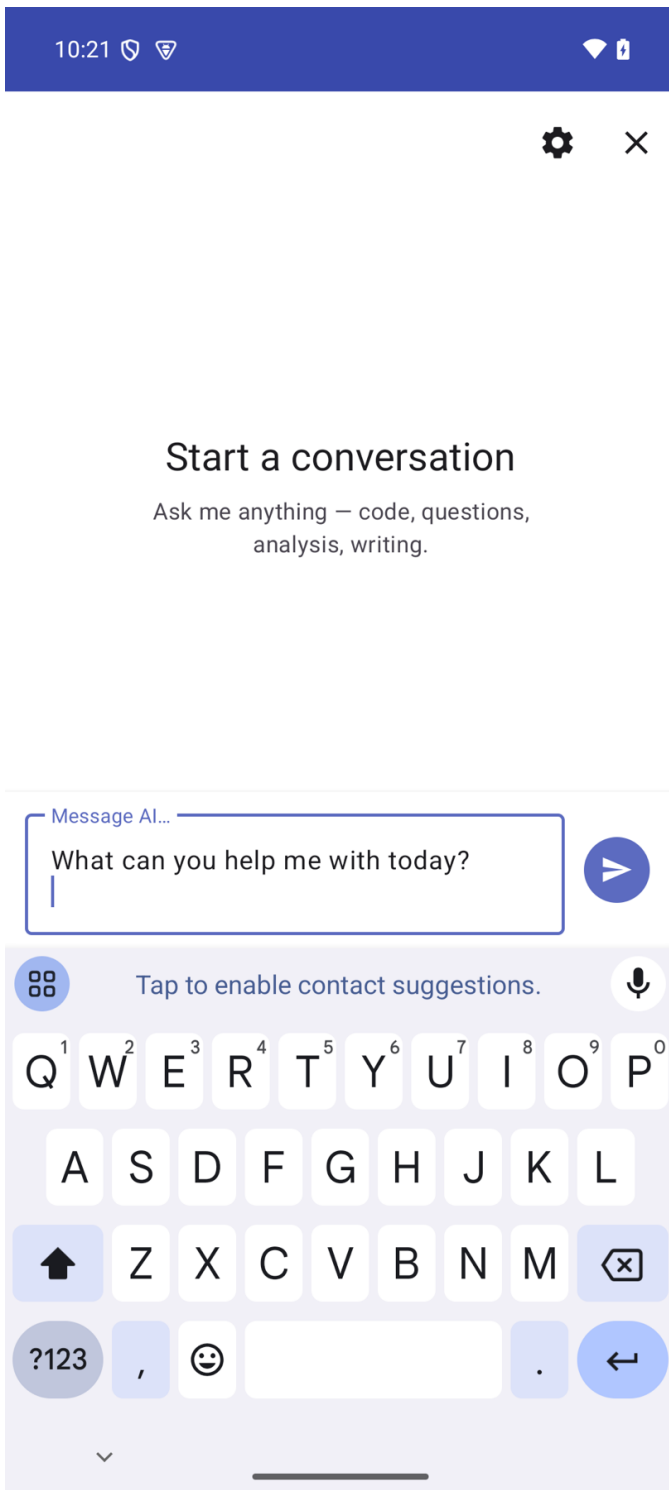


Fig. 1. The ChatFragment UI mid-conversation, showing a completed streamed assistant response rendered in an AI bubble alongside the preceding user message via ChatAdapter.